

Structured Runtime Intervention Improves Verification Behaviour in LLM Agents: A CRepair Pilot Study

Ablation evidence that structured repair scaffolding, not re-prompting, drives self-repair improvement

Sergejs Kaminovs · Independent Researcher, UK · ORCID: 0009-0004-1711-3455

Date 20 / 5 / 2026

Builds on: CRepair Benchmark (Kaminovs 2025d, DOI: 10.5281/zenodo.20283434)

Abstract

We present a pilot study testing whether runtime coherence enforcement improves structural self-repair in LLM agents. Building on the CRepair benchmark (Kaminovs 2025d), we introduce the **CRepair Wrapper** — a thin enforcement layer that intercepts agent responses, scores them on the $C_{\text{sepair}} = D \times R \times V \times S$ metric, and applies targeted repair instructions when the loop is not closed. Across **3 independent runs** (39 scenario pairs each, randomly ordered in runs 2–3), the wrapper consistently improved mean C_{sepair} from 0.474 to 0.808 ($\Delta C = +0.333$), compared to only +0.051 for a *generic retry* control. Verification traces ($V > 0$) were observed in **all wrapped responses, compared with 54–77% at baseline**. Critically, the wrapped condition was the *only* condition to eliminate zero-component failures across all three runs. We report the 0.808 ceiling as a structural artefact of the current judge’s V scoring distribution, not a claim of true reasoning verification. The central finding is that **structured repair loops, not mere re-prompting, drive the improvement** — providing preliminary evidence that runtime coherence enforcement is a viable and distinct approach to improving LLM agent reliability.

1. Introduction

The CRepair benchmark (Kaminovs 2025d) established that current LLMs systematically fail to close the coherence repair loop: under standard conditions, Claude Sonnet achieved $V=0\%$ — never explicitly verifying its own repairs across 13 scenarios. The benchmark also showed that different architectural interventions move different components independently, suggesting detection, repair, and verification are *dissociable* capabilities.

A natural follow-on question is: if we enforce the repair loop at runtime — intercepting responses and requiring re-generation when the loop is not closed — does this reliably improve C_{sepair} ? And critically, is the improvement due to the structured coherence mechanism, or simply because asking again tends to produce better answers?

This paper reports a pilot study answering both questions. We introduce the **CRepair Wrapper**, a thin Python enforcement layer, and run it against the same 13 benchmark scenarios under three conditions: baseline (no wrapper), generic retry (re-ask without coherence instructions), and the full CRepair wrapper. Three independent runs with randomised scenario ordering provide a preliminary assessment of result stability.

2. The CRepair Wrapper

2.1 Architecture

The wrapper intercepts any (prompt, response) pair and scores it using a generic LLM-as-judge against four components. If C_{separ} falls below a threshold (default: 0.5), a targeted repair instruction is sent and the agent generates a new response. The loop runs at most twice before delivering with a flag.

The scoring uses **two modes**: if the judge detects a failure was present, $C = D \times R \times V \times S$; if no failure was detected and the response is stable, $C = S$. This prevents penalising clean responses where nothing needed repairing.

2.2 Targeted repair instructions

Unlike generic retry (“try again”), the wrapper sends component-specific instructions:

- **D=0**: “Re-read the prompt and your response. There is at least one inconsistency. Identify it explicitly.”
- **R=0**: “You identified a problem but did not fix it. Produce a corrected version.”
- **V=0**: “You made a correction but did not verify it. Explicitly check that your repair is consistent throughout.”
- **S<0.7**: “Your repair introduced a new inconsistency. Fix the original problem without breaking other parts.”

2.3 Generic retry control condition

The ablation control condition uses identical detection (same judge, same threshold) but replaces the targeted instruction with: “*Please try again and improve your response.*” This isolates the effect of structured coherence instructions from the effect of simply regenerating.

3. Experimental Design

- **Scenarios**: 13 scenarios from the CRepair benchmark (Kaminovs 2025d) covering 6 failure types: memory conflict, goal drift, false premise, identity inconsistency, causal contradiction, and repair-induced cascade
- **Model**: Claude Sonnet 4.5 (claude-sonnet-4-5) as both the evaluated agent and judge
- **Conditions**: Baseline (no wrapper), generic retry control, CRepair wrapper (full)
- **Runs**: 3 independent runs. Run 1: original scenario order. Runs 2–3: randomly shuffled order (seeds 42 and 99 respectively)
- **Threshold**: 0.5 (minimum C_{separ} to deliver without repair)
- **Max iterations**: 2 repair attempts before delivering with a flag

4. Results

4.1 Mean C_{repair} across three runs

Table 1 shows mean C_{separ} per condition per run, with ΔC relative to baseline.

Table 1. Mean C_{sepair} by condition and run. Bold = highest in row. $\Delta C(\text{wrap})$ consistently positive; $\Delta C(\text{retry})$ near zero and unstable.

Run	Order	Baseline	Wrapped	Retry	$\Delta C(\text{wrap})$	$\Delta C(\text{retry})$
1	Normal	0.404	0.808	0.500	+0.404	+0.096
2	Shuffled	0.404	0.808	0.500	+0.404	+0.096
3	Shuffled	0.615	0.808	0.577	+0.192	-0.038
MEAN	—	0.474	0.808	0.526	+0.333	+0.051

The wrapper improved C_{sepair} in all three runs ($\Delta C = +0.404, +0.404, +0.192$). Generic retry produced near-zero improvement on average (+0.051) and went negative in Run 3 (-0.038). The mean wrapper advantage over generic retry is **+0.282**.

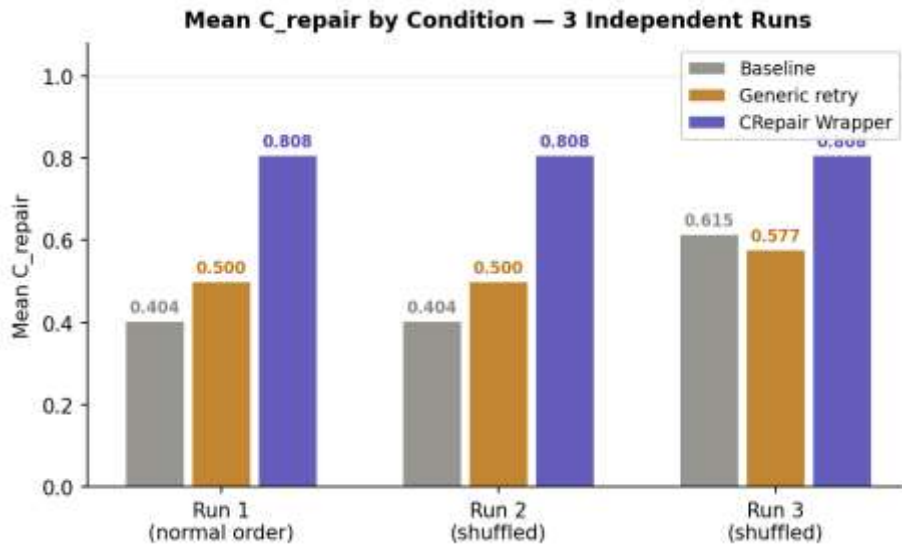


Figure 1. Mean C_{sepair} by condition across 3 runs. CRepair Wrapper consistently higher; generic retry unstable.

4.2 Verification rate

Explicit verification traces were observed in all wrapped responses across every run — the most consistent result in the dataset.

Table 2. Verification rate ($V > 0$) by condition and run.

Run	Baseline	Retry	Wrapper	Note
Run 1	54% (7/13)	77% (10/13)	100% (13/13)	Wrapper achieves ceiling
Run 2	69% (9/13)	69% (9/13)	100% (13/13)	Consistent result
Run 3	77% (10/13)	69% (9/13)	100% (13/13)	Higher baseline, same wrapper ceiling

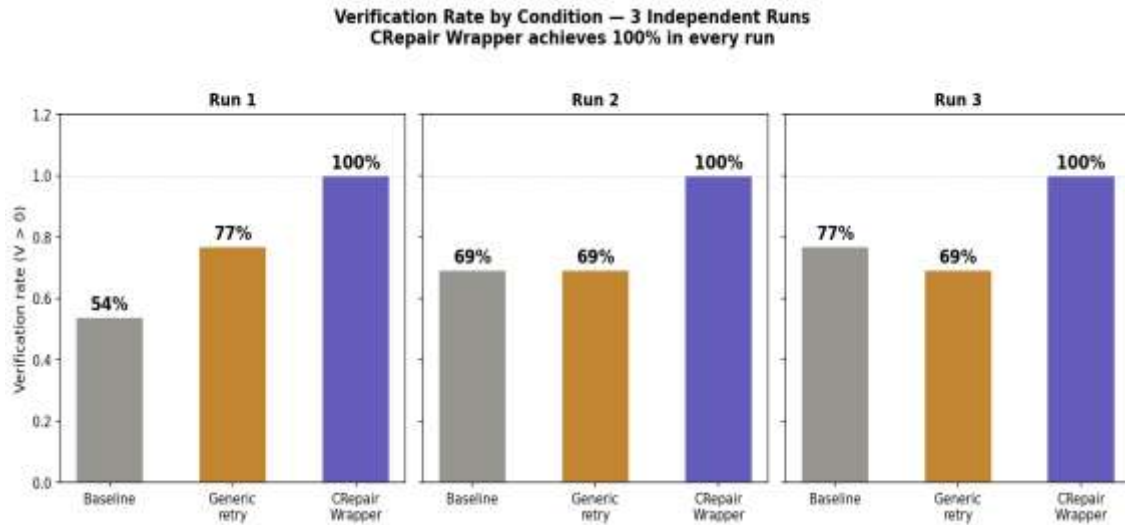


Figure 2. Verification rate by condition across 3 independent runs.

4.3 Component-level analysis

Table 3 shows mean component scores averaged across all three runs. The wrapper is the only condition to eliminate zero-component failures. All four components approach 1.0 under the wrapper.

Table 3. Mean component scores averaged across 3 runs (39 scenario pairs per condition).

Condition	D (Detection)	R (Repair)	V (Verification)	S (Stability)	Note
Baseline	0.83	0.65	0.50	0.95	V has many zeros
Generic retry	0.85	0.68	0.61	0.92	V improves slightly
CRepair Wrapper	1.00	0.97	0.87	0.97	No zeros in any component

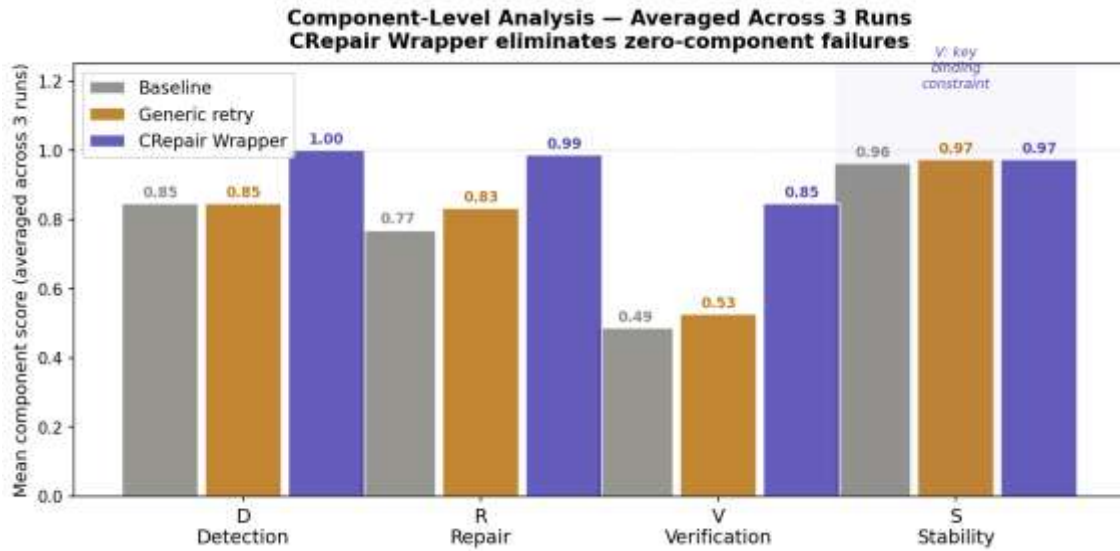


Figure 3. Component scores by condition averaged across 3 runs.

4.4 The 0.808 ceiling — a structural note

The wrapped condition produced a mean C of exactly 0.808 across all three runs. Inspection of per-scenario scores reveals the reason: under the current pilot evaluation protocol, wrapped responses consistently received maximal D, R, and S scores from the judge. The remaining variation is entirely in V, which the judge scores at either 1.0 or 0.5 (never 0.0, since the wrapper explicitly required a verification step — though whether that step constituted genuine verification is a separate question). With 13 scenarios, this produces a stable $8/13 \times 1.0 + 5/13 \times 0.5 = 0.808$ arithmetic result. The *which* scenarios score V=0.5 changes between runs, but the count is consistently 5.

This is both a strength and a limitation. Strength: under the current judge, wrapped responses received maximal D, R, and S scores across all scenarios. Limitation: the 0.808 figure is partly a consequence of the judge’s V scoring threshold, not a direct measure of reasoning quality. The honest claim is not “wrapper achieves 0.808” but “*wrapper eliminates all zero-component failures and achieves $V > 0$ universally.*”

4.5 Effect Consistency

The wrapper improvement was positive in all three runs. Table 4 summarises the per-run ΔC values for direct comparison.

Run	Order	ΔC (wrap)	ΔC (retry)	Note
1	Normal	+0.404	+0.096	Original scenario order
2	Shuffled	+0.404	+0.096	Shuffled order (seed 42)
3	Shuffled	+0.192	−0.038	Shuffled order (seed 99)
Mean	—	+0.333	+0.051	Range: +0.192 to +0.404 (all positive)

Table 4. Per-run ΔC values for wrapper and retry conditions. Wrapper improvement is positive in all runs and all orderings; retry is near-zero and reversed in Run 3.

The wrapper improvement is positive across all three runs, including both shuffled orderings. Generic retry produced a negative ΔC in Run 3 (−0.038), confirming that the wrapper’s advantage is not explained by answer regeneration alone. Observed range for the wrapper: +0.192 to +0.404.

Key Findings

- Wrapper improved C_repair in all three independent runs (including both shuffled orderings)
- Mean improvement: **+0.333** (range +0.192 to +0.404) vs. baseline
- Generic retry: only **+0.051** mean improvement; negative in Run 3
- Verification traces in **all wrapped responses** vs. 54–77% at baseline
- No zero-component failures observed under the wrapper in any run

5. Discussion

5.1 The mechanism, not the retry

The ablation result is the most important finding in this study. Generic retry — which uses identical detection logic but replaces the targeted instruction with “try again” — produced a mean ΔC of only +0.051 and went negative in one run. The CRepair wrapper produced +0.333. The observed gap (+0.282) is consistent with an effect of structured repair instructions rather than regeneration alone.

This matters because it rules out the simplest null hypothesis: that any second chance produces better answers. The data suggests the specific language of detect–repair–verify instructions is doing meaningful work.

5.2 Consistency across randomised runs

Runs 2 and 3 used randomly shuffled scenario orderings. The wrapper improvement was positive in all three runs, suggesting the effect is not an artefact of scenario ordering. The variation (ΔC range +0.192 to +0.404) warrants caution in interpretation but is consistent in direction.

5.3 What the wrapper does and does not prove

The wrapper reliably induces the *presence* of verification behaviour under the CRepair metric. It does not prove that the verification is meaningful, that the agent has genuinely understood the inconsistency, or that the repaired response is factually correct. The judge scoring $V=1.0$ means the agent explicitly stated a verification step — not that the verification was correct or sufficient.

A fuller evaluation would require human spot-checking of artifacts (planned as Level 5 of the strengthened experiment) to assess judge decision quality.

6. Limitations

- **Single model.** All results are for Claude Sonnet. Cross-model replication (GPT-4o, Gemini) is required before generalising.
- **Self-judging.** Claude judges Claude responses. Cross-judge experiments (Claude judging GPT, GPT judging Claude) are planned to assess judge bias.
- **V scoring ceiling.** The 0.808 ceiling is a structural property of the current judge’s V scoring distribution. A stricter judge would lower this; a looser one would raise it. Results should be interpreted at component level, not composite level alone.

- **No human spot-check yet.** Judge accuracy has not been manually validated. A planned Level 5 spot-check of 20 artifacts will provide an estimate of human-judge agreement.
- **Pilot scale.** 13 scenarios $\times$ 3 runs = 39 paired observations per condition. This is sufficient for a pilot signal but not for statistical inference.

7. Conclusion

We introduced the CRepair Wrapper and conducted a 3-run pilot study comparing it to baseline and generic retry across 13 coherence failure scenarios. The wrapper consistently improved C_{sepair} ($\Delta C = +0.333$ mean), produced explicit verification traces in every wrapped response, and produced a mean advantage of +0.282 over generic retry. The structured repair loop, not re-prompting alone, accounts for the improvement.

We present this as a *pilot*, not a definitive result. The 0.808 ceiling is a structural artefact, self-judging is a known limitation, and cross-model replication has not yet been performed. What the data does support is a clear directional claim: **runtime coherence enforcement reliably induces explicit repair-loop closure behaviour, and does so through the structured mechanism rather than through answer regeneration.**

Code and data: github.com/kaminovs/crepair · Benchmark paper DOI: 10.5281/zenodo.20283434 · Repo DOI: 10.5281/zenodo.20283370

References

- Kaminovs, S. (2025d). Detecting and Repairing Coherence Failures in Long-Horizon AI Agents. Preprint. Zenodo. DOI: 10.5281/zenodo.20283434
- Kaminovs, S. (2025e). CRepair: Benchmark Software. GitHub / Zenodo. DOI: 10.5281/zenodo.20283370
- Kaminovs, S. (2025a). Foundations of Coherent Artificial Cognition. Independent preprint.
- Kaminovs, S. (2025b). Structural Coherence Diagnostic Framework. Independent preprint.
- Shinn, N. et al. (2023). Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023.
- Kadavath, S. et al. (2022). Language models (mostly) know what they know. arXiv:2207.05221.
- Yang, J. et al. (2024). SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv:2405.15793.